

Distributed Systems from First Principles

A distributed system contains independently failing components that communicate with uncertain delays. Correct design states what remains true during retries, crashes, and partitions. Start from invariants and failure boundaries, then select replication and coordination mechanisms that support the promised behavior.

For: Engineers who know APIs and databases and want to understand replication, consistency, partitions, retries, and coordination without relying on slogans.

Suggested pace: 4-6 weeks of failure-oriented study | Backend fundamentals required

What you will be able to do

- Distinguish local failure, communication failure, and an unknown operation outcome.
- Explain consistency choices in terms of user-visible reads and writes.
- Design idempotent processing with durable identities and bounded retries.
- Reason about leadership, fencing, replication, and recovery without claiming universal exactly-once behavior.

Build the right mental model

A timeout is ambiguous

The remote operation may not have started, may still be running, or may have completed with a lost response. The client cannot infer which solely from elapsed time. Durable operation identity and status reconciliation make retries safer.

Consistency is a contract

Describe which writes a read must observe and under what failure conditions. Eventual convergence, read-your-writes, and linearizable operations have different costs and guarantees. Do not choose a database category as a substitute for stating the needed semantics.

Ownership must survive failures

A lease can expire while a paused worker later resumes. A fencing token or generation checked by the protected resource can prevent stale owners from committing. Merely believing that a new leader exists does not stop an old process from acting.

Study with evidence.

For every design, trace success, lost response, duplicate delivery, process crash, and partition. State the durable boundary and what a client can safely retry. Use a small local simulation before studying a larger product's architecture.

Week 1: Failures and identities

Study RPC timeouts, retries, idempotency keys, backoff, and deadlines. Build a request table that stores a logical operation's status and result. Test duplicate requests with identical and conflicting payloads.

Deliverable and check: A retry contract that distinguishes safe replay from a new business action.

Week 2: Replication and reads

Compare leader-based replication, asynchronous replicas, and quorum concepts. Describe stale reads and failover behavior. Learn that replication does not replace backups or protect against every correlated failure.

Deliverable and check: A read/write timeline with explicit consistency and data-loss assumptions.

Weeks 3-4: Coordination and recovery

Study logs, consensus at a conceptual level, leases, epochs, and fencing. Analyze a paused worker that resumes after replacement. Add durable checkpoints and replay to a simple job processor.

Deliverable and check: A recovery scenario where stale work cannot overwrite the current owner's result.

Weeks 5-6: Operate the system

Add bounded queues, backpressure, observability, and failure injection. Define service-level goals, recovery objectives, and a runbook. Evaluate which guarantees remain possible during a network partition.

Deliverable and check: A design note with invariants, failure matrix, capacity limits, and a tested recovery path.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

A job finishes, but its acknowledgement is lost

A worker updates a database and crashes before acknowledging its queue message. The broker later redelivers the same job. The system must avoid applying a business effect twice.

A worked approach

1. Give the job a stable ID. In the same database transaction as the business update, record that the job has been applied, using a uniqueness constraint. If the transaction never commits, redelivery can safely attempt it again.
2. On redelivery, inspect the durable job record and return the stored outcome rather than repeating the effect. Acknowledge only after the transaction reaches the promised durable boundary.
3. If the effect is an external API call rather than a local transaction, preserve a provider idempotency key and reconcile ambiguous outcomes. Do not claim that the database deduplication record atomically controls an unrelated remote service.

Expected result

The local database effect is deduplicated under the stated transaction contract. External effects require their own coordination or reconciliation strategy; the design names that boundary instead of hiding it under 'exactly once.'

Verify the result

- Is the deduplication record committed with the local effect?
- Can the same job ID be reused with a different payload?
- What happens when the acknowledgement or provider response is lost?

Extend the exercise

Add a worker lease and simulate a long pause. Require the destination to check a monotonically increasing fencing value before accepting a write from a resumed stale worker.

Make the learning stick.

Does replication guarantee no data loss?

Only within its stated durability, acknowledgement, and failure model. Asynchronous replicas can lag, and correlated failures or logical corruption can affect all copies. Explain the recovery-point objective and backup strategy separately.

Does CAP mean choose any two forever?

That slogan is misleading. During a partition, a system cannot simultaneously provide the relevant strong consistency and availability guarantees for every operation. Discuss the operation, failure conditions, and actual contract rather than assigning one permanent label.

Is exactly-once delivery the same as exactly-once effect?

No. Delivery, processing, and an external business effect are different boundaries. Transactions and deduplication can establish useful scoped guarantees, but each destination and retention window must be considered explicitly.

Avoid these traps

- Treating a timeout as evidence that nothing happened.
- Using leases without preventing stale owners from writing.
- Promising end-to-end exactly-once effects from a queue setting alone.

Ready for the next step?

- Every important operation has a named invariant.
- Retries preserve a durable logical identity.
- Read consistency and failure behavior are explicit.
- Recovery tests cover lost responses and stale workers.

Reliable next reads

- [MIT distributed systems course](#)
- [Raft consensus reference](#)
- [Azure architecture patterns](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.